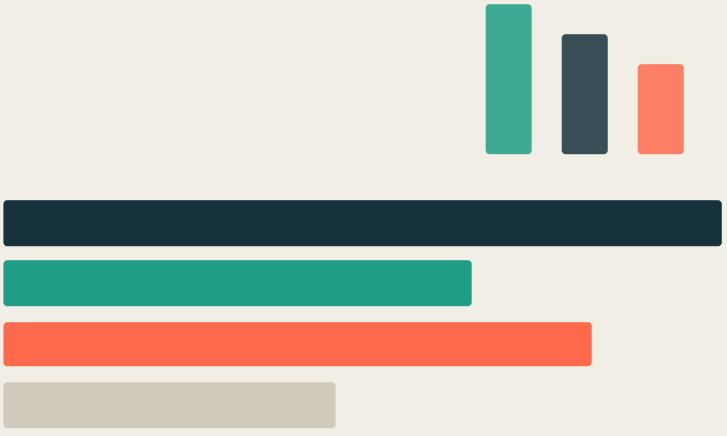
2026

PORTFOLIO REVIEW

R Shiny Project Portfolio

Four Shiny applications reviewed for architecture, engineering decisions, and the details that signal technical seniority.



Daniel Amah

ML / MLOps Engineer — R & Shiny

akandaniel.netlify.app

github.com/AkanimohOD19A

OVERVIEW

What signals seniority in a Shiny project

Anyone can make a Shiny app run. Seniority shows in how it is structured, how it fails, and whether anyone else could maintain it. Each project in this review is read against the same six signals.

- **Modular architecture** — Shiny modules with explicit inputs and outputs instead of one long app.R
- **Deliberate reactivity** — `eventReactive`, `debounce`, gated recomputation
- **A real data layer** — parameterised SQL and aggregation in the database
- **Performance awareness** — background workers, debounced inputs, work kept off the main thread
- **Production concerns** — testable core, CI, lockfiles, graceful failure
- **Design judgment** — rules in pure functions, state as data, policy separated from UI

THE FOUR PROJECTS IN THIS REVIEW

01 **BMI Health Calculator** p.03
Foundations — gated reactivity, validation, bslib layout

02 **Real-Time Transaction Fraud Monitoring Dashboard** p.05
Streaming scores, async workers, live thresholds, triage queue

03 **Merchant Compliance & KYC Review Console** p.07
State machine, policy as pure functions, audit trail, async screening

04 **Credit Risk & Vintage Analytics Explorer** p.09
Parameterised SQL layer, vintage curves, what-if scenarios

SIGNALS AT A GLANCE

	01 BMI	02 Fraud	03 KYC	04 Credit
Shiny modules	–	3	6	4
Async (future / promises)	–	•	•	–
Database layer, parameterised SQL	–	–	–	•
Pure-function core + testthat	–	•	•	–
renv lockfile	–	•	•	–
CI (GitHub Actions)	–	•	•	–
DESCRIPTION file (package metadata)	–	•	•	–

PROJECT 01

BMI Health Calculator

Foundations: a single-file app that sets the reactive-graph and input-validation patterns every later project builds on.

R Shiny | bslib (flatly) | shinyWidgets | bsicons | dplyr | shinyapps.io

TIER

Fundamentals — app #1 in a planned 45-app tiered roadmap

STRUCTURE

Single-file app.R (UI + server)

STATUS

Open source; README documents shinyapps.io deployment

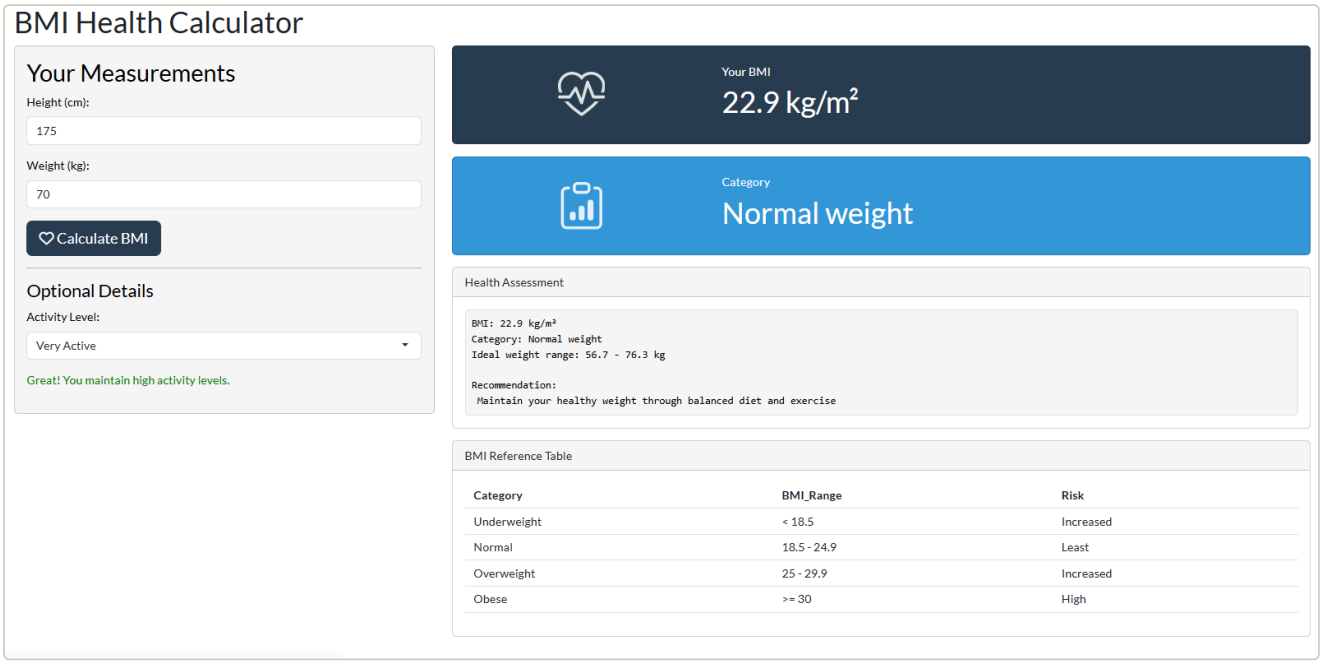
THE PROBLEM

Calculating BMI is trivial; making it trustworthy is not. The app has to refuse nonsense input, explain what the number means, and give guidance without the clutter of typical calculator sites.

WHAT I BUILT

A single-page calculator: height and weight go in, and on request the app returns BMI, a category, an ideal-weight range derived from height (18.5–24.9 BMI), and a plain-language recommendation. Results are shown in colour-coded `value_box()` cards alongside a reference table.

It is deliberately small. Its job in the portfolio is to establish clean habits (gated computation, validated inputs, modern bslib layout) before the architecture gets harder in projects 02–04.



The calculator after pressing Calculate: value boxes, health assessment and reference table

ARCHITECTURE & KEY SHINY COMPONENTS

- **Gated computation** — one `eventReactive(input$calculate)` produces a single result list that feeds every output; nothing recalculates while the user is still typing
- **Defensive inputs** — `req()` stops on missing values, then `validate(need())` enforces physical ranges and shows the message in the UI instead of throwing an error
- **Client-side UI logic** — `conditionalPanel()` reacts to the activity-level select in the browser, with no server round trip
- **Modern layout** — `bs_theme(bootswatch = "flatly")`, `value_box()` and `card()` with bsicons showcases
- **One source of truth** — category and recommendation are both derived from the same BMI value with `case_when()`, so they cannot disagree
- **Deployment** — published to shinyapps.io per the project README

SIGNALS OF TECHNICAL SENIORITY IN THIS PROJECT

- ✓ Computation is gated to an explicit user action, not left to fire on every keystroke
- ✓ Bad input produces a readable message rather than a stack trace
- ✓ Scope is honest: the README lists "refactor into R/ modules" as the next step, which projects 02–04 then deliver

FROM APP.R — VALIDATED, GATED CALCULATION

```
bmi_data <- eventReactive(input$calculate, {
  req(input$height_cm, input$weight_kg)

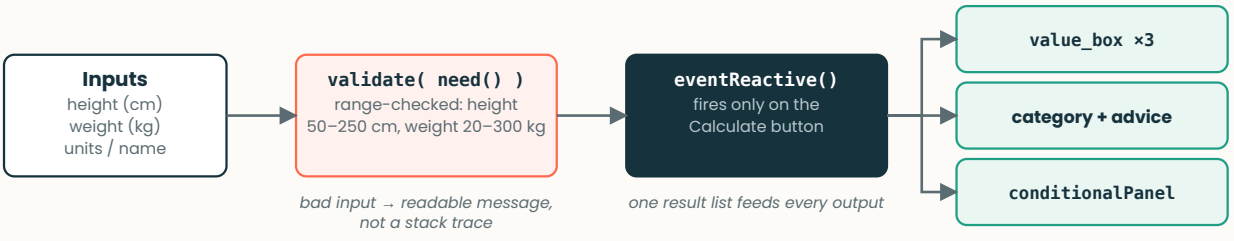
  validate(
    need(input$height_cm > 50 && input$height_cm < 250,
         "Height must be between 50 and 250 cm"),
    need(input$weight_kg > 10 && input$weight_kg < 300,
         "Weight must be between 10 and 300 kg")
  )
  height_m <- input$height_cm / 100
  bmi      <- input$weight_kg / (height_m ^ 2)
  ...
```

1
eventReactive feeding every result output

2
validate() range rules guarding the calculation

45
apps in the planned tiered portfolio roadmap

ARCHITECTURE — REACTIVE GRAPH: ONE TRIGGER, VALIDATED INPUTS



PROJECT 02

Real-Time Transaction Fraud Monitoring Dashboard

A streaming risk-scoring console: scores arrive on a timer, analysts tune alert thresholds live, and flagged items land in a triage queue.

R Shiny | Shiny modules | future / promises | reactivePoll | DT | testthat | renv | GitHub Actions

STRUCTURE

DESCRIPTION-based layout with 3 Shiny modules and a pure scoring core

DATA

Simulated feed — one function, isolated as the swap point for a real stream

STATUS

Open source

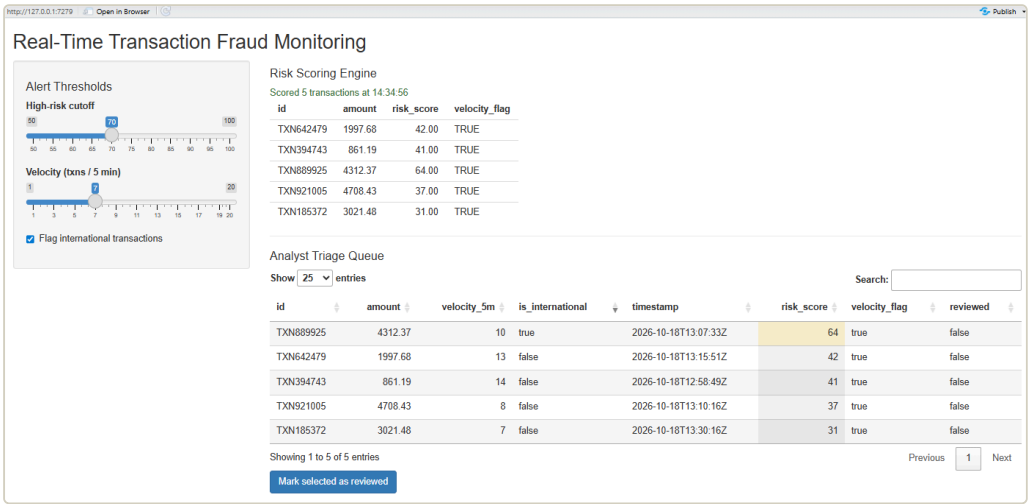
THE PROBLEM

A fraud desk needs new transactions scored the moment they arrive and a way to decide what deserves human attention, with thresholds analysts can change without touching code, and a UI that never freezes while scoring runs.

WHAT I BUILT

A `reactivePoll()` pulls a fresh batch of transactions every two seconds. Each batch is scored in a background R worker, so the interface stays responsive. A thresholds module (risk cutoff, velocity per five minutes, international flag) returns a reactive list that the triage module applies to the scored batch.

The triage table is a `DT` grid with row colouring keyed to the analyst's own cutoff, and a "mark as reviewed" action. The transaction source is simulated, and it lives in one function (`simulate_transactions()`) so a real feed can replace it without touching the modules.



The running dashboard: threshold sidebar, scoring engine output and the analyst triage queue

ARCHITECTURE & KEY SHINY COMPONENTS

- **Module contract** — three `moduleServer()` modules wired in `app.R` by passing reactives: transactions → scored → triage, plus thresholds. No module reaches into another's namespace
- **Async with a failure path** — `future()` with `%...>%` to store results and `%...!%` to turn scoring errors into UI state instead of a dead session
- **Startup guard** — `plan(multisession, workers = 2)` is verified by a probe future and falls back to sequential if workers cannot start
- **Pure core** — `risk_scoring.R` (score, flag, threshold) has no Shiny dependency and is covered by testthat: score bounds, the international boost, velocity flag, threshold edges
- **Reproducibility** — `renv.lock`, a `DESCRIPTION` declaring Imports and Suggests, and a GitHub Actions `R-CMD-check` workflow
- **Reactive styling** — `formatStyle(styleInterval())` shades risk scores relative to the analyst's current cutoff

SIGNALS OF TECHNICAL SENIORITY IN THIS PROJECT

- ✓ Reactive wiring is explicit: data flows through module arguments and return values, not globals
- ✓ Failure is designed for: a scoring error becomes visible status text, not a crashed R session
- ✓ Business logic can be tested without launching the app

FROM MOD_RISK_SCORING.R — NON-BLOCKING SCORING WITH AN ERROR PATH

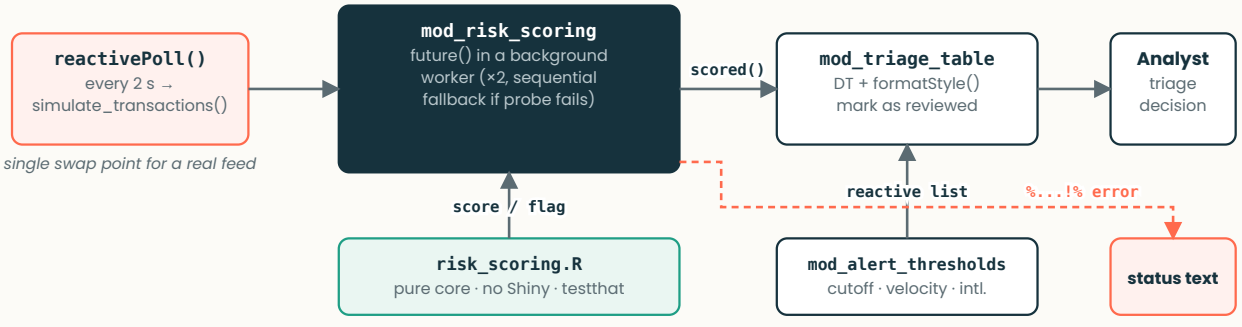
```
future({
  score_transactions(txns)
}) %...>% (function(result) {
  scored(result)
}) %...!% (function(err) {
  # Fail gracefully: expose the error as state the UI can render
  scored(list(error = TRUE, message = conditionMessage(err)))
})
```

3
Shiny modules wired through reactives

2 s
poll interval for the incoming batch

2
background workers, with sequential fallback

ARCHITECTURE — MODULE AND DATA FLOW, INCLUDING THE FAILURE PATH (DASHED)



Wired in app.R by passing reactives; no module reaches into another's namespace.

PROJECT 03

Merchant Compliance & KYC Review Console

A case-management console for merchant KYC: prioritised queue, case detail, async sanctions screening, justified decisions and an audit trail.

R Shiny | bs4Dash | Shiny modules | DT | future_promise | testthat | renv | CI: Windows + Ubuntu

STRUCTURE

DESCRIPTION-based layout, 6 Shiny modules, one pure business-rules file

DATA

12 seeded demo cases and a sample sanctions list

STATUS

Open source

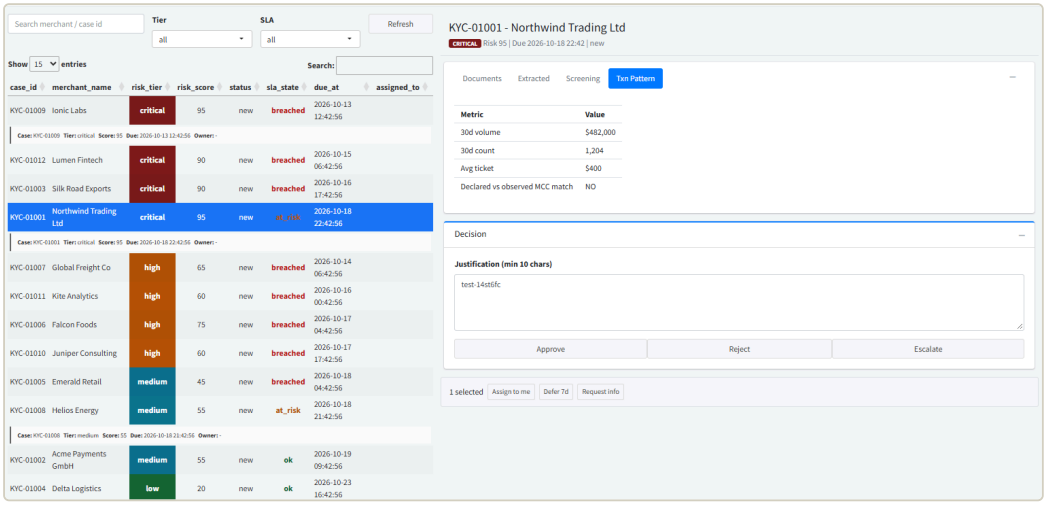
THE PROBLEM

Compliance analysts reviewing merchant onboarding and periodic-review cases need a queue that surfaces the riskiest and most overdue work first, and decisions that are justified, rule-checked and traceable afterwards.

WHAT I BUILT

The queue sorts by risk tier, then SLA breach, then due date, with tier and SLA filters and a debounced free-text search. Selecting a case opens detail tabs (documents, extracted fields, screening, transaction pattern). Analysts approve, reject or escalate through a decision panel that demands a written justification.

Sanctions re-screening runs asynchronously against a simulated slow API. Every state change is written to an audit trail. All policy lives in pure functions, separate from the UI.



Case queue with tier and SLA colouring, expandable rows, case detail and the decision panel

ARCHITECTURE & KEY SHINY COMPONENTS

- **Explicit state machine** — `next_status(from, action)` is a lookup table of 11 legal transitions across 5 statuses. Anything else is an error, and one `apply_transition()` function is the only place cases mutate
- **Policy as pure functions** — `compliance_rules.R`: risk score with documented weights capped at 100, tier thresholds, token-overlap sanctions matching, SLA due-time with pause, decision validation
- **Guardrails in code** — justification of at least 10 characters; no approval while a sanctions hit is open; breached high or critical cases must escalate
- **Audit by construction** — `audit_append()` schema-checks and returns a new data frame. The caller rebinds the reactive value, so history is never edited in place
- **Responsiveness** — `debounce(400)` on search; sanctions screening via `future_promise()` with a `%...!%` error handler
- **Delivery** — test files for rules, SLA, queue state and audit; a CI matrix on Windows and Ubuntu restoring from `renv.lock`

SIGNALS OF TECHNICAL SENIORITY IN THIS PROJECT

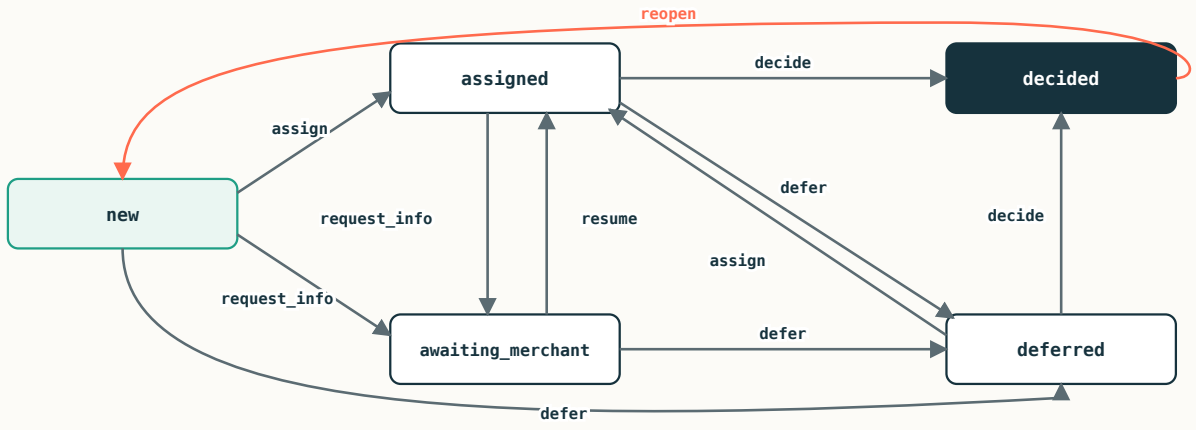
- ✓ Compliance logic is deterministic and documented, so a weight can be explained to an auditor
- ✓ Workflow is modelled as a transition table, not if-statements scattered across observers
- ✓ Policy (pure functions) is separated from presentation (modules), which is what makes it unit-testable

6
Shiny modules, one pure rules file

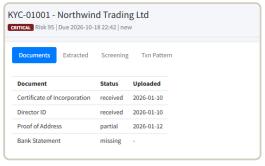
11
legal case-state transitions across 5 statuses

4
SLA tiers: 24 / 48 / 120 / 240 hours

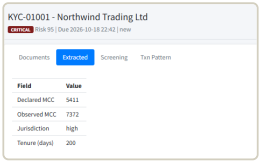
ARCHITECTURE — THE CASE WORKFLOW AS A STATE MACHINE — THE WHOLE OF NEXT_STATUS()



11 legal (status, action) pairs across 5 statuses. Anything else raises "Illegal transition".



Documents tab



Extracted fields tab

PROJECT 04

Credit Risk & Vintage Analytics Explorer

A self-serve explorer over a loan-level SQLite store: shared filters, SQL-side aggregation, vintage curves and a what-if panel.

R Shiny | bslib (page_sidebar) | DBI / RSQLite | SQL | plotly | bsicons

DATA

50,000 synthetic loans, up to 36 months on book, built by a seeded script

STRUCTURE

4 Shiny modules plus a dedicated database layer (R/db.R)

STATUS

Open source; the .db is git-ignored and rebuilt by data/seed_db.R

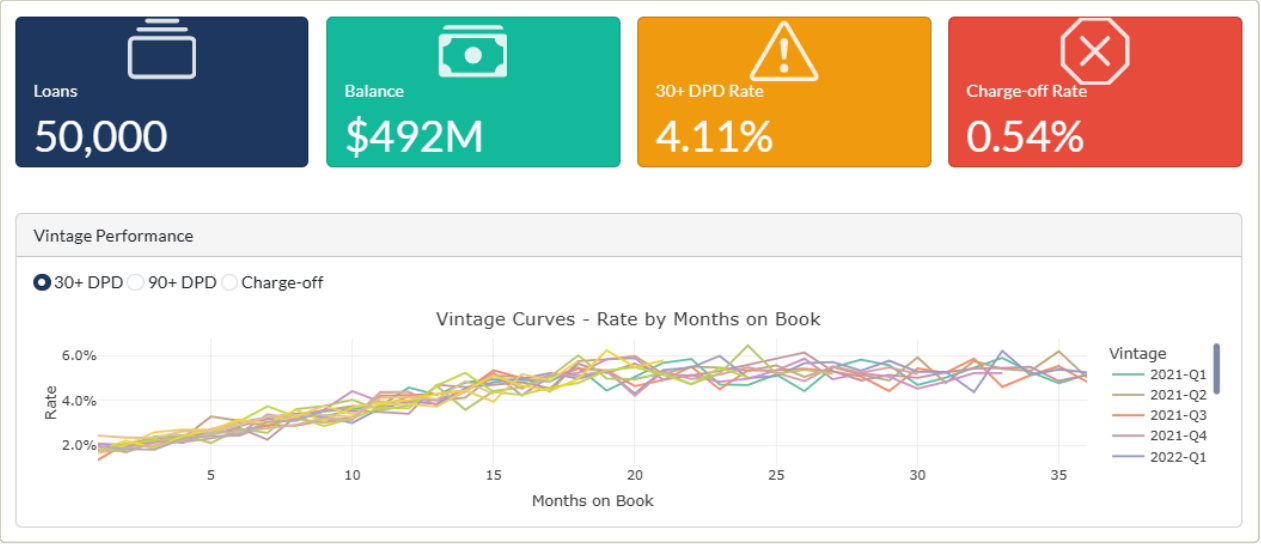
THE PROBLEM

Risk analysts keep asking for the same cuts: delinquency and charge-off by vintage, product, region and risk tier. A self-serve tool replaces a queue of ad-hoc requests, provided it stays fast on a loan-level dataset.

WHAT I BUILT

Four multi-select filters feed one shared, parameterised WHERE clause. That single filter drives the KPI cards (loans, balance, 30+ DPD rate, charge-off rate), the vintage-curve chart (30+ DPD, 90+ DPD or charge-off by months on book, in plotly) and a loan drill-down table.

A what-if panel takes FICO, LTV and DTI limits and shows base against scenario figures when the user presses Recompute. It is an approximation: the FICO cut uses a normal-distribution estimate rather than re-querying loans.



KPI row and vintage curves (30+ DPD rate by months on book, one line per origination quarter)

ARCHITECTURE & KEY SHINY COMPONENTS

- **A real data layer** — `R/db.R` is the only file that writes SQL. Modules receive reactives and data frames, never connections or query strings
- **Safe by construction** — `build_filter_clause()` emits `?` placeholders plus a params list, so no user input is concatenated into SQL
- **Least privilege** — the connection opens with `SQLITE_RO` and is closed in `session$onSessionEnded()`
- **Aggregate in the database** — KPIs and vintage rates are computed in SQL; R only receives and plots the result
- **Deliberate recompute** — `eventReactive()` on the curves and on the what-if button keeps expensive paths user-triggered
- **Reproducible data** — `set.seed(2024)` seed script builds the database with six indexes; the binary is not committed

SIGNALS OF TECHNICAL SENIORITY IN THIS PROJECT

- ✓ Large data stays in the database instead of being loaded into R
- ✓ The query layer is injection-safe and isolated, so the backend can change without touching the modules
- ✓ The dataset is reproducible from code, with no binary committed to Git

FROM R/DB.R — BUILDING A PARAMETERISED FILTER (EXCERPT, TRIMMED)

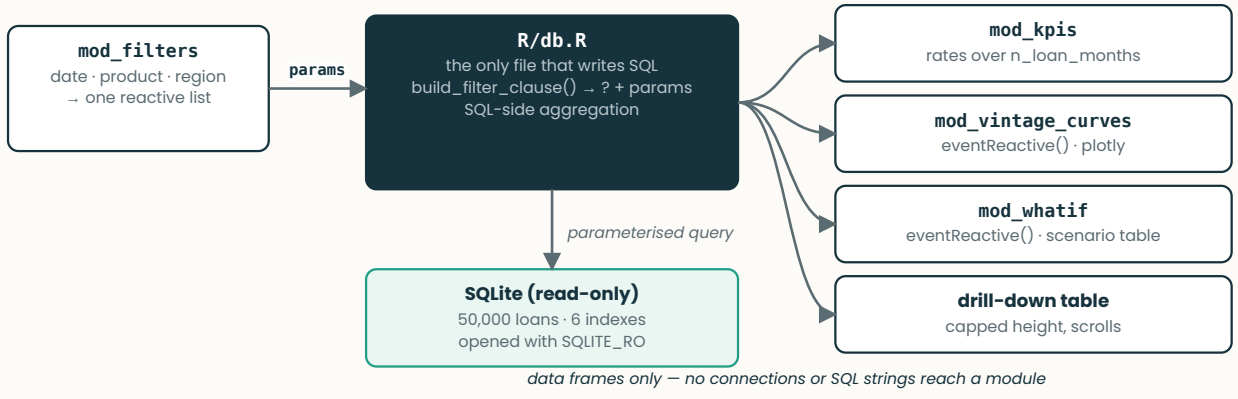
```
if (length(products)) {
  clauses <- c(clauses, sprintf("l.product IN (%s)",
                paste(rep("?", length(products)), collapse = ",")))
  params  <- c(params, as.list(products))
}
...
list(
  sql    = paste("WHERE", paste(clauses, collapse = " AND ")),
  params = params
)
```

50,000
loans in the seeded portfolio

36
maximum months on book simulated

6
indexes on the join and filter columns

ARCHITECTURE — LAYERING: FILTERS → QUERY LAYER → READ-ONLY DATABASE → MODULES



SUMMARY

Patterns across the portfolio

- **Policy separated from presentation** — fraud scoring and KYC rules are plain R functions that can be tested without Shiny
- **State as data** — the KYC workflow is a transition table, and the audit trail is an append-only data frame
- **Heavy work off the UI thread** — scoring and sanctions screening run in workers with error handlers
- **Data access isolated** — all SQL lives in one file with parameterised queries and a read-only connection
- **Reproducible by default** — renv lockfiles, seeded data scripts, CI workflows
- **Progression you can follow** — from validated inputs (01) to modules and async (02), to workflow and audit (03), to a database layer (04)

WHAT I WOULD BUILD NEXT

- **Real authentication and persistence (03)** — replace the hard-coded analyst identity with real sign-in and write the audit trail to a database, not memory
- **A live feed (02)** — swap `simulate_transactions()` for a Kafka or API consumer behind the same interface
- **End-to-end tests (all)** — add `shinytest2` flows; it is already declared in the Suggests of 02 and 03
- **Profiling and load (04)** — profile with `profvis` and test the query layer against Postgres at larger volume

WHERE TO FIND THE CODE

All four projects live in one public repository, one folder each, with the per-project READMEs, tests and workflows referred to in this document.

github.com/AkanimohOD19A/shiny-mastery
repository

akandaniel.netlify.app
portfolio site

WHAT THIS PORTFOLIO SHOWS

- ✓ **Production Shiny structure:** modules with explicit reactive contracts, a database layer, async with error paths (projects 02–04)
- ✓ **Testable, reviewable code:** pure business rules under testthat, renv lockfiles and GitHub Actions checks (02, 03)
- ✓ **Domain judgment:** defensible rules, audit trails and least-privilege data access, the concerns of regulated domains such as finance and compliance
- ✓ **Open to conversation:** the code, tests and READMEs are public. Happy to walk through any of the four projects